

# DevOps and Cloud Computing



Cloud

VS

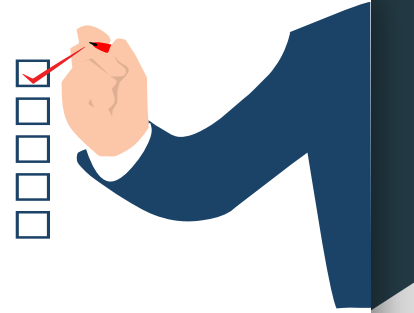


DevOps

**Dr Benabderrezak**  
**Researcher in Cyber Security**  
**Full-stack Web developer**

# Work Plan

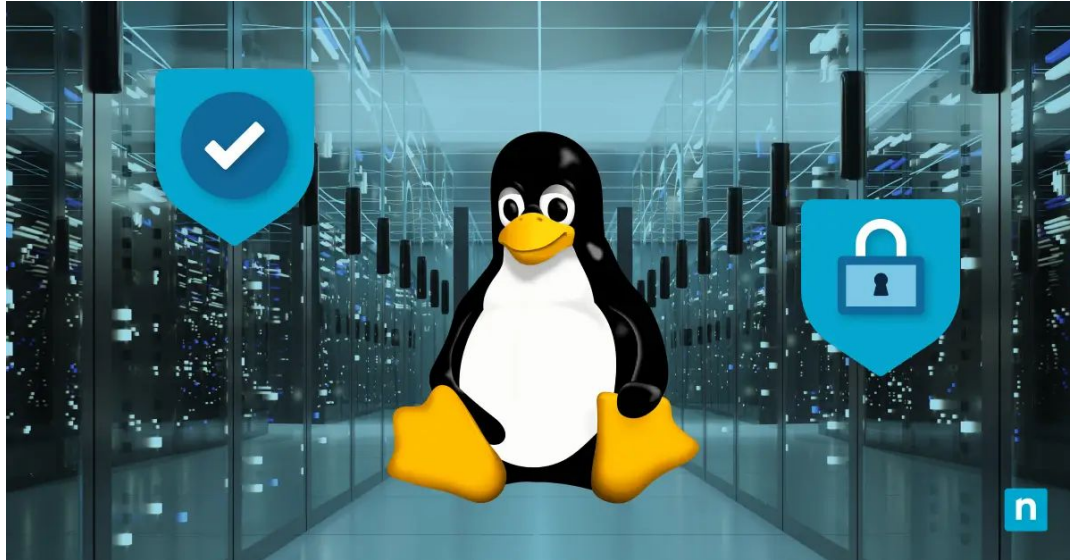
1. Linux Basics
2. Virtualization
3. Cloud Computing Environment & Applications
4. Introduction to DevOps
5. Version Control With Git
6. Continuous Integration and Continuous Delivery (CI/CD)
7. Infrastructure as Code (IaC)
8. Containerization with Docker



# Linux Basics

## Why it matters ?

Linux is the backbone (العمود الفقري) of most servers and DevOps tools.



## Linux Key concepts

- File Management
- User Management
- Networking Basics
- Process Management

## File Management : Know how to navigate files , search content

| Command | Description                      |
|---------|----------------------------------|
| ls      | List files in a directory.       |
| cd      | Change directory.                |
| mkdir   | Create a new directory.          |
| touch   | Create an empty file.            |
| cp      | Copy files or directories.       |
| mv      | Move or rename files.            |
| rm      | Remove files or directories.     |
| find    | Search for files or directories. |
| du      | Check disk usage of a directory. |
| df      | Check available disk space.      |

| <b>Command</b> | <b>Description</b>                            |
|----------------|-----------------------------------------------|
| pwd            | Print the current directory.                  |
| stat           | Display file or directory information.        |
| ln             | Create symbolic or hard links.                |
| basename       | Extract the filename from a path.             |
| dirname        | Extract the directory name from a path.       |
| chmod          | Change file permissions.                      |
| chown          | Change file ownership.                        |
| cat            | Display file content.                         |
| more           | View file content page by page.               |
| less           | View file content (with backward navigation). |

| <b>Command</b> | <b>Description</b>                            |
|----------------|-----------------------------------------------|
| head           | Show the first 10 lines of a file.            |
| tail           | Show the last 10 lines of a file.             |
| split          | Split files into smaller parts.               |
| wc             | Count words, lines, and characters in a file. |
| tar            | Archive files into a single file.             |
| gzip           | Compress files.                               |
| gunzip         | Decompress .gz files.                         |
| zip            | Create compressed .zip files.                 |
| unzip          | Extract .zip files.                           |
| file           | Determine a file's type.                      |

**User Management** : Manage users and permissions with adduser or chmod.

|         |                                            |
|---------|--------------------------------------------|
| whoami  | Display the current user.                  |
| id      | Show user and group IDs.                   |
| users   | List logged-in users.                      |
| who     | Show who is logged in.                     |
| w       | Show logged-in users and their activities. |
| passwd  | Change the user's password.                |
| su      | Switch to another user.                    |
| adduser | Add a new user.                            |
| useradd | Add a new user (alternative).              |
| usermod | Modify an existing user.                   |

|          |                                             |
|----------|---------------------------------------------|
| deluser  | Remove a user.                              |
| userdel  | Remove a user (alternative).                |
| groupadd | Create a new group.                         |
| groupdel | Delete a group.                             |
| groups   | Show groups for the current user.           |
| chage    | Manage user password expiration.            |
| finger   | Display information about a user.           |
| last     | Show the login history of users.            |
| sudo     | Execute commands with superuser privileges. |
| visudo   | Edit the sudoers file safely.               |

|               |                                        |
|---------------|----------------------------------------|
| logname       | Display the current user's login name. |
| whoami        | Confirm the logged-in user.            |
| getent        | Retrieve user account entries.         |
| gpsswd        | Manage group passwords.                |
| newgrp        | Switch to a different group.           |
| usermod       | Add a user to a group.                 |
| pkill -u      | Kill all processes for a user.         |
| smbpasswd     | Change Samba user password.            |
| net usershare | Manage Samba user shares.              |
| hostnamectl   | Change or query the system's hostname. |

**Networking Basics** : Check connectivity with ping or open ports with netstat.

|            |                                              |
|------------|----------------------------------------------|
| ifconfig   | Display or configure network interfaces.     |
| ip         | Manage IP addresses and routes.              |
| ping       | Check the connection to another host.        |
| traceroute | Trace the route packets take to a host.      |
| netstat    | Show network connections and statistics.     |
| ss         | Display socket statistics.                   |
| nslookup   | Query DNS for domain name or IP information. |
| dig        | Perform DNS queries.                         |
| curl       | Transfer data from or to a server.           |
| wget       | Download files from the internet.            |

|         |                                             |
|---------|---------------------------------------------|
| scp     | Copy files over SSH.                        |
| rsync   | Synchronize files between systems.          |
| ftp     | Connect to an FTP server.                   |
| telnet  | Connect to a remote host.                   |
| ssh     | Log into a remote machine securely.         |
| arp     | Display or modify the ARP table.            |
| route   | Show or change the routing table.           |
| nmap    | Scan a network for open ports.              |
| tcpdump | Capture and analyze network packets.        |
| ethtool | Display or modify Ethernet device settings. |

## Process Management

|         |                                           |
|---------|-------------------------------------------|
| ps      | Show running processes.                   |
| top     | Display real-time process info.           |
| htop    | Interactive process manager.              |
| kill    | Terminate a process by PID.               |
| killall | Terminate processes by name.              |
| pgrep   | Find process IDs by name.                 |
| pkill   | Kill processes by name.                   |
| bg      | Resume a suspended job in the background. |
| fg      | Bring a job to the foreground.            |
| jobs    | List active jobs.                         |

# **Virtualization**

## What is Virtualization ?

- Running multiple "**virtual computers**" on a single physical machine
- **Example** : Running Windows inside your Linux PC using VirtualBox



## Why Use Virtualization ?

- Save money by using fewer machines
- Easily test software in different environments

## How Does It Work ?

**Hypervisor** :Software like VMware or VirtualBox manages virtual machines.

## Types :

- Fully virtual (entire computer is simulated)
- Partially virtual (shares some hardware).

# **Chapter 2**

## **Cloud Computing Environment**

## What is Cloud Computing ?

- Using **remote servers** (instead of your own) to store files, run apps, or analyze data.
- **Example** : Google Drive or Dropbox.



shutterstock.com · 315594821

## How is the Cloud Used ( Models ) :

- SaaS (e.g., Google Docs) : You use software online
- PaaS (e.g., Heroku) : Developers build apps online
- IaaS (e.g., AWS EC2) : Rent virtual machines

## Why Use the Cloud ?

- It's flexible, scalable, and you pay only for what you use

# **Chapter 3**

## **Cloud Applications**

## Where the Cloud Helps

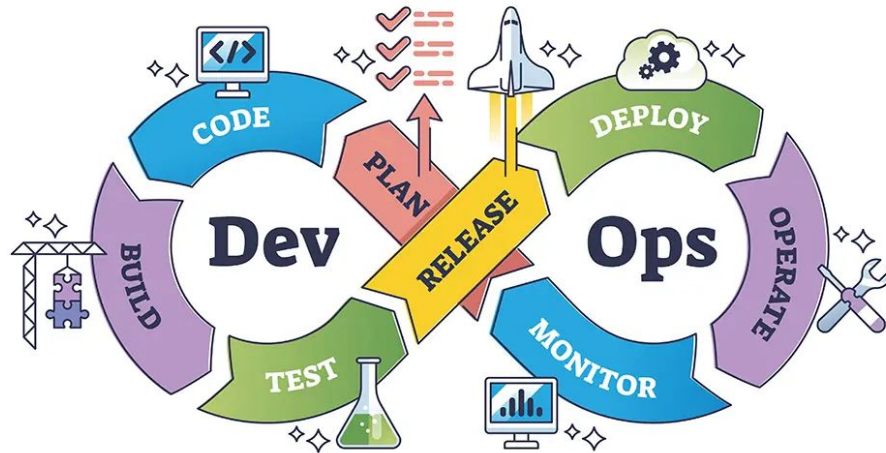
- **Safety** : Backups and encryption keep data secure
- **Storage** : Services like AWS S3 hold massive amounts of data
- **Management** : Tools like AWS CloudWatch monitor usage
- **Resource Allocation** : The system automatically adjusts resources when demand changes

# **Chapter 4**

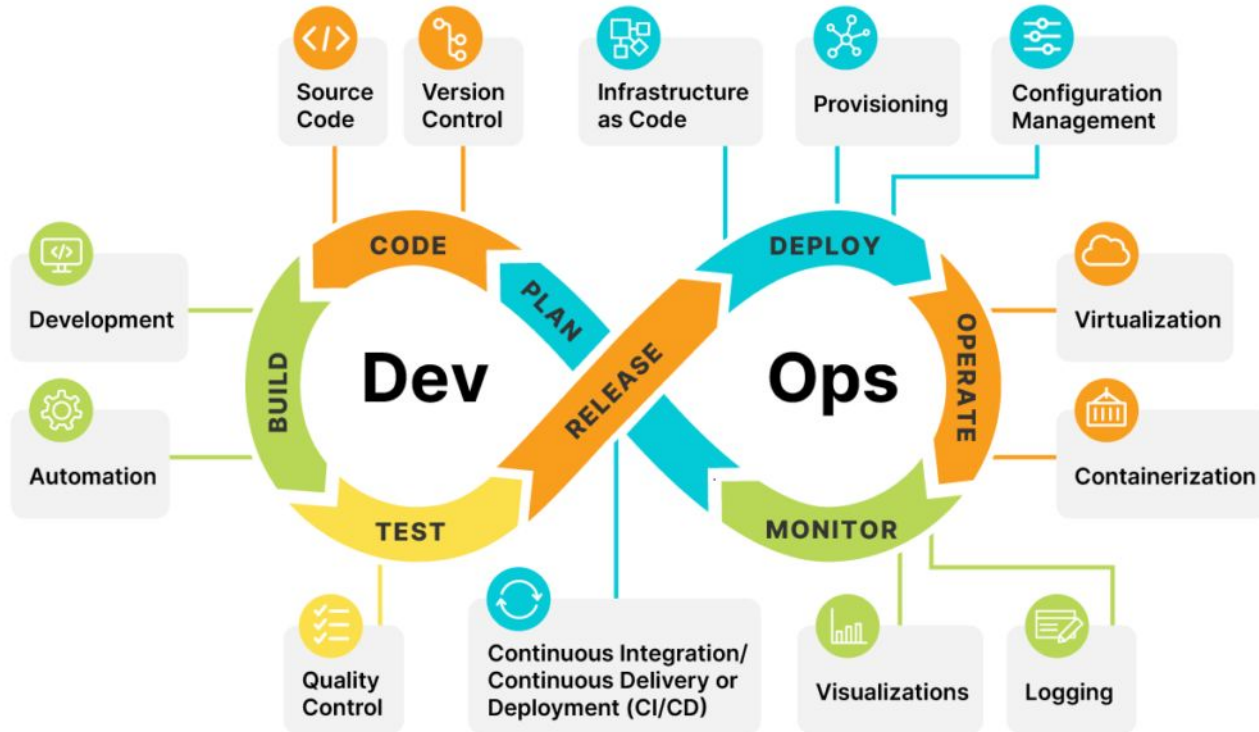
## **Introduction to DevOps**

## What is DevOps ?

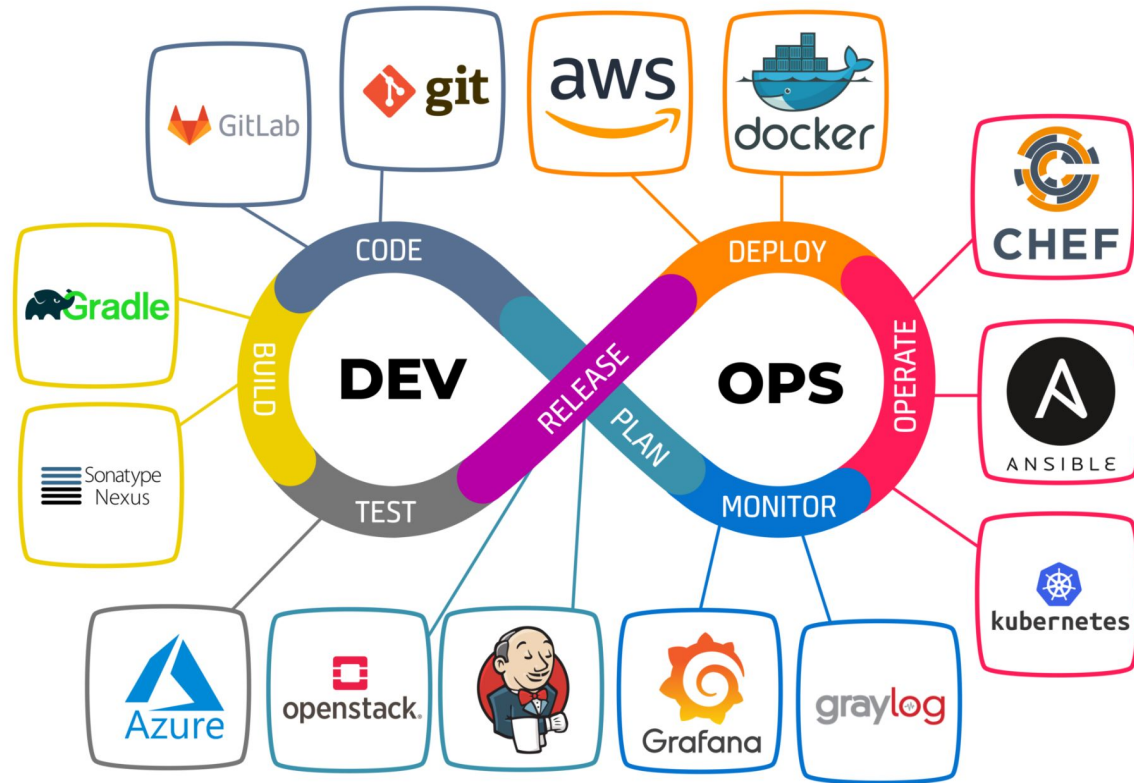
- A way of working where developers and operations teams collaborate
- **Goal** : Build software faster and make it reliable



# What is DevOps ?



# DevOps Tools



## Why DevOps Matters :

- Quicker releases, fewer bugs, better teamwork
- **Example** : Automating code deployment instead of doing it manually



# Chapter 5

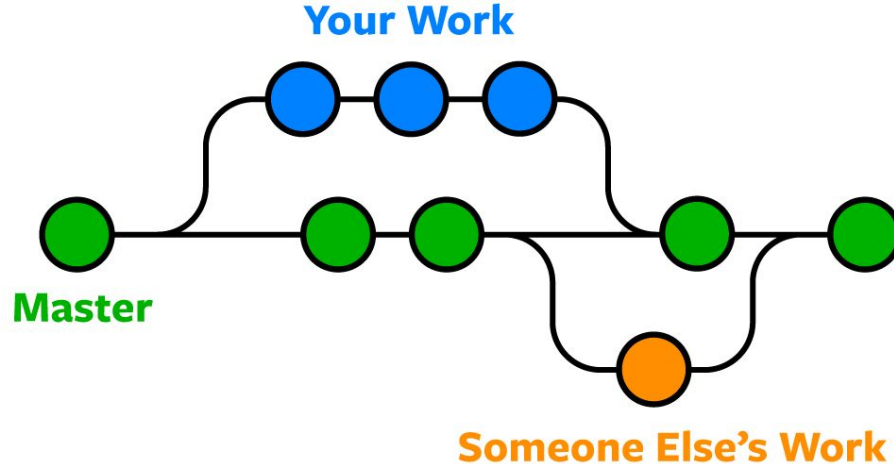
## Version Control with Git



**git**

## What is Git ?

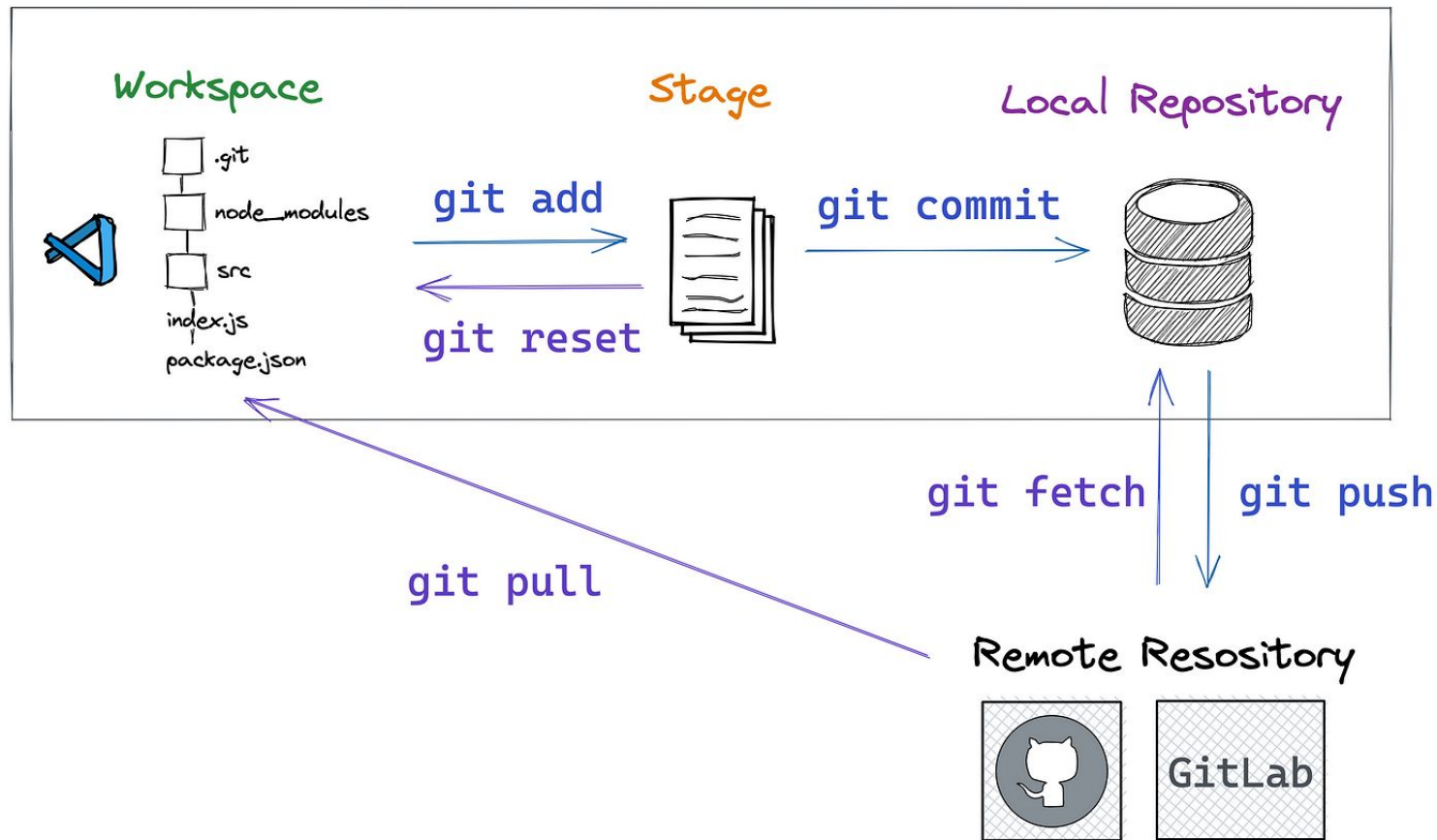
- A tool to track changes in code and work with teams.
- Used to : Save code, undo mistakes, and share work with others.



## Basic Git Commands

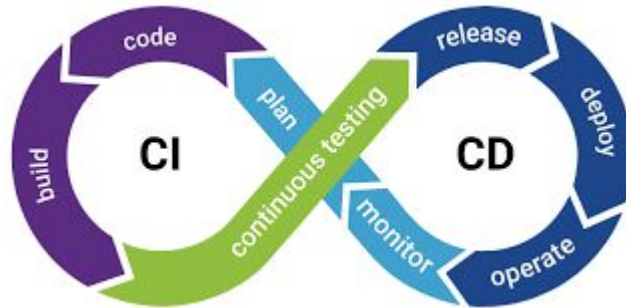
- Save changes : **git add** and **git commit**
- Share your code : **git push**
- Get code from remote repository : **git pull**
- To Work in parallel, Create branches for different tasks : **git checkout -b branch\_name**

# Local



# Chapter 6

## Continuous Integration & Delivery (CI/CD)



## What is CI/CD ?

- **CI** : Automatically test code whenever someone changes it
- **CD** : Automatically send the tested code to production.

## Why It's Useful

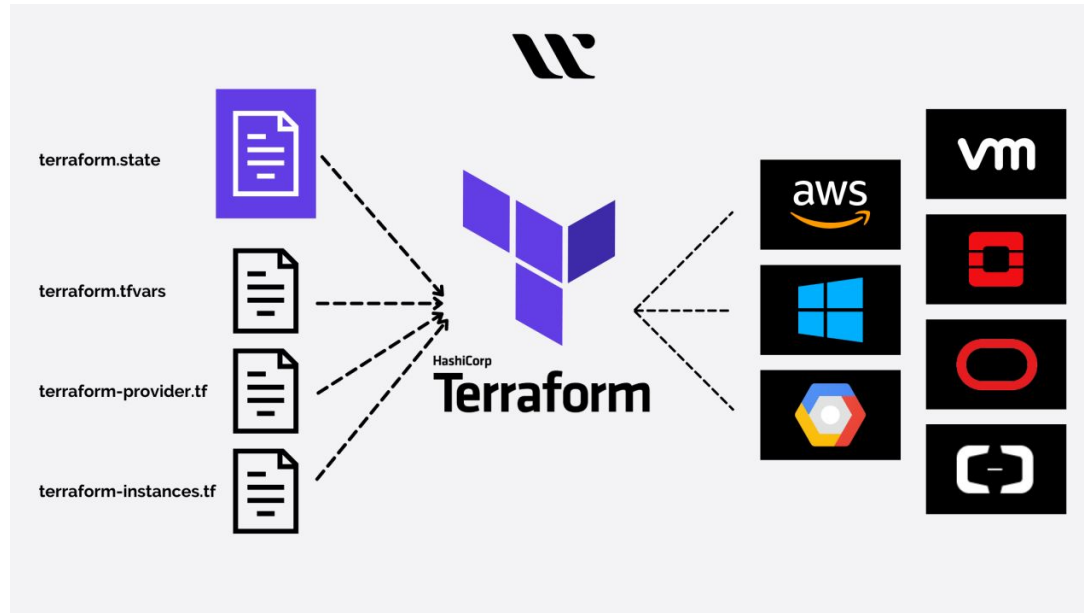
- Saves time and avoids mistakes.
- **Example** : Use **Jenkins** to run automated tests whenever you push code.

# Chapter 7

## Infrastructure as Code (IaC)

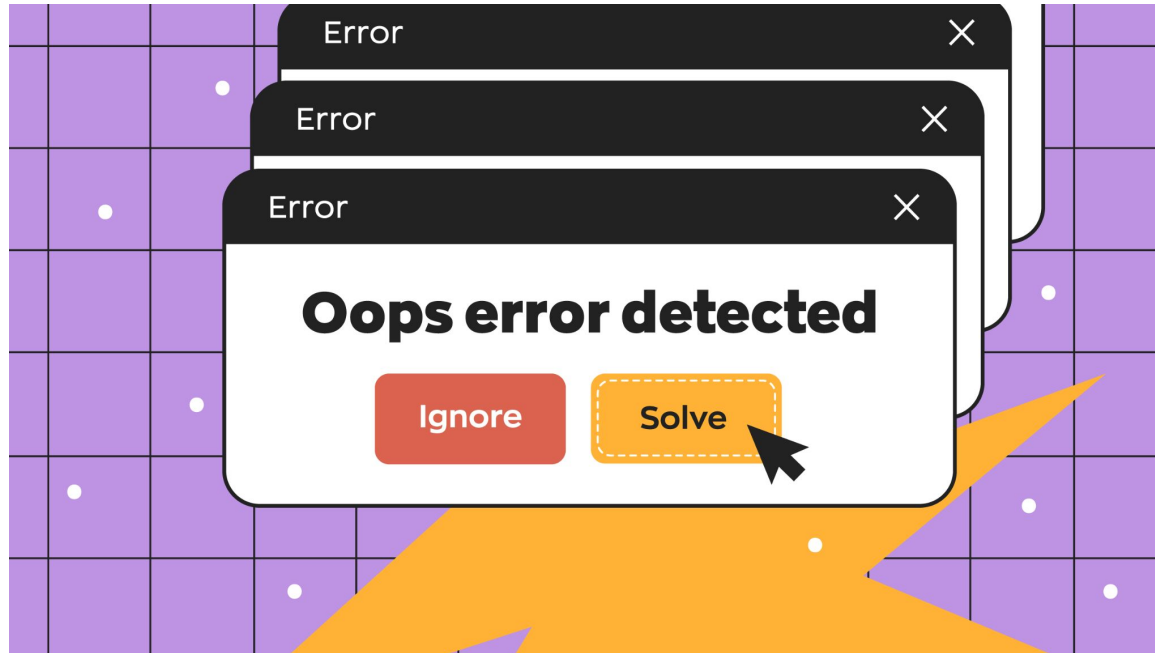
## What is IaC ?

- Managing servers and configurations using code instead of manual setup.
- Example : **Terraform** or Ansible to deploy servers.



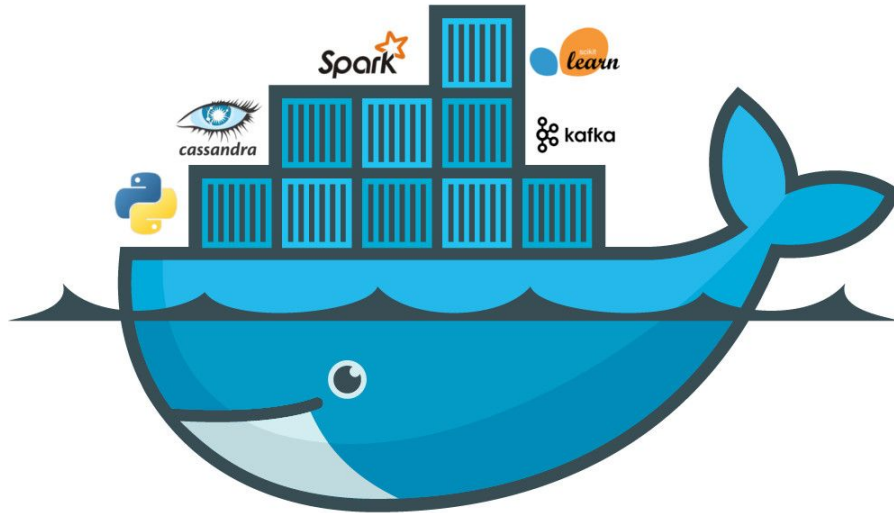
## Why Use It ?

- Easier to make changes, reuse configurations, and reduce errors



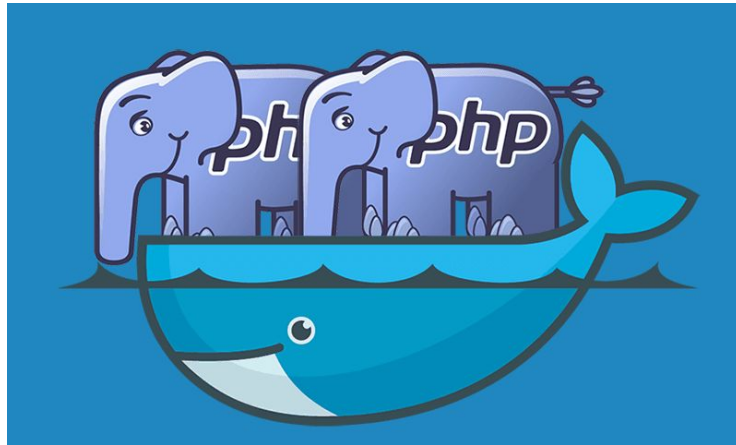
# Chapter 8

## Containerization with Docker

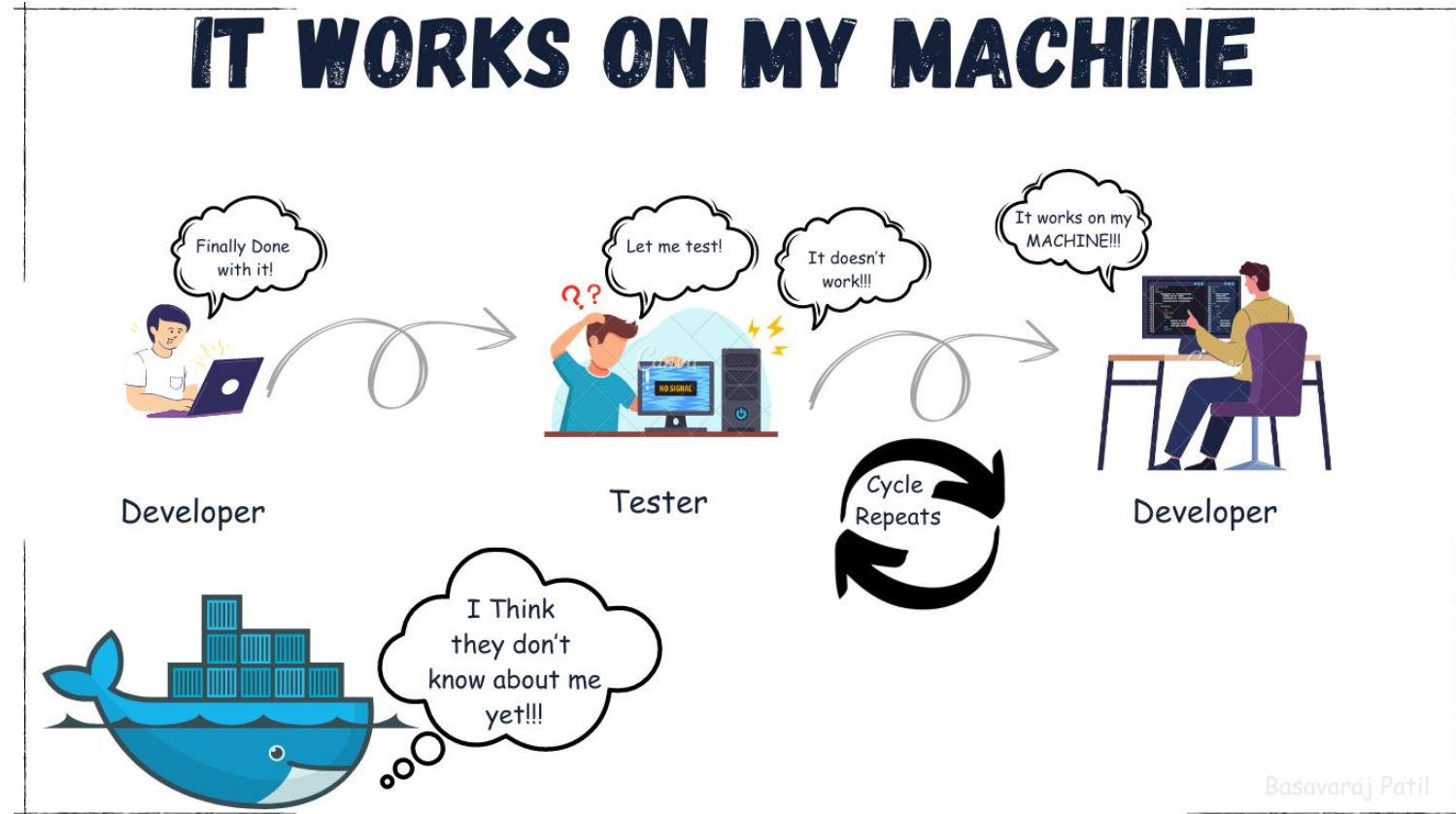


## What is Docker ?

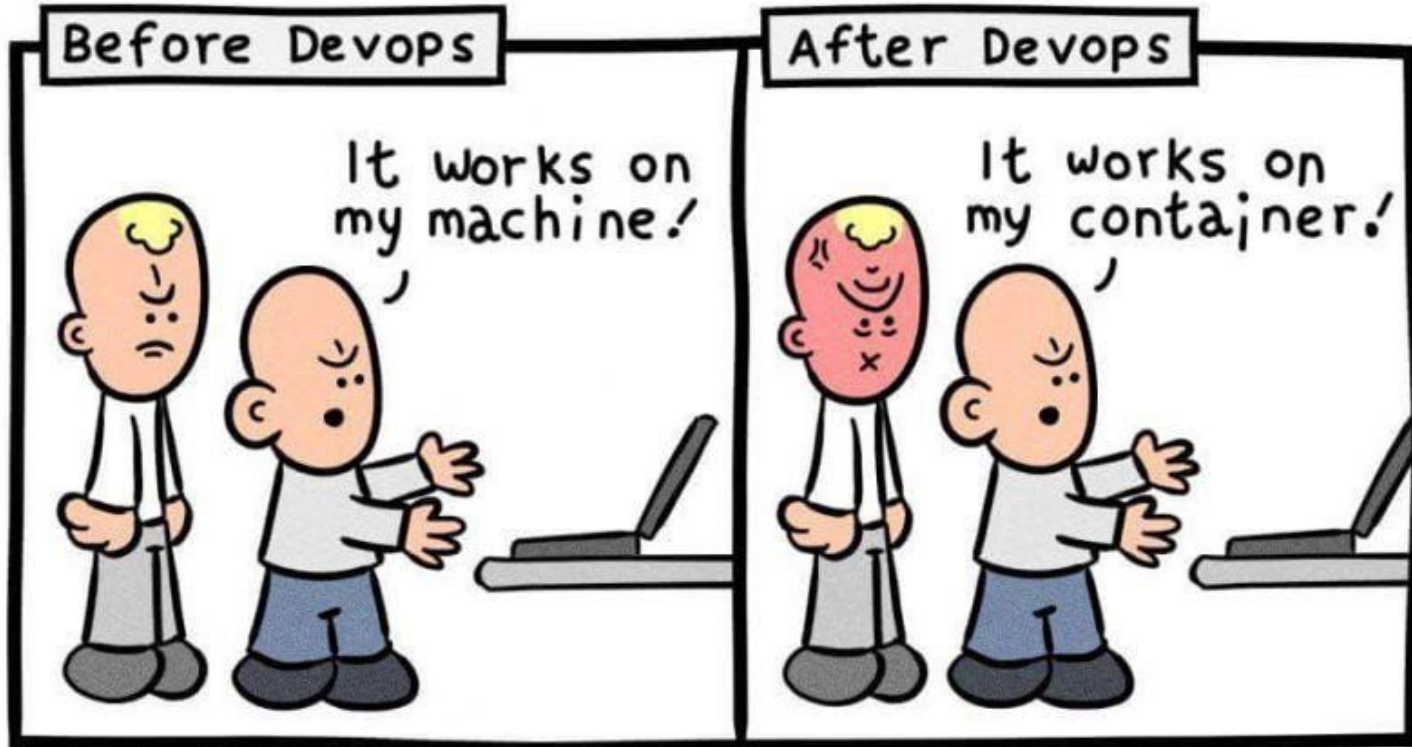
- A tool to package apps with everything they need to run anywhere.
- **Example** : Running a **PHP** app in a **Docker container** ensures it works the same way on any machine.



## Why Use Docker ? : Avoid the “it works on my machine” problem

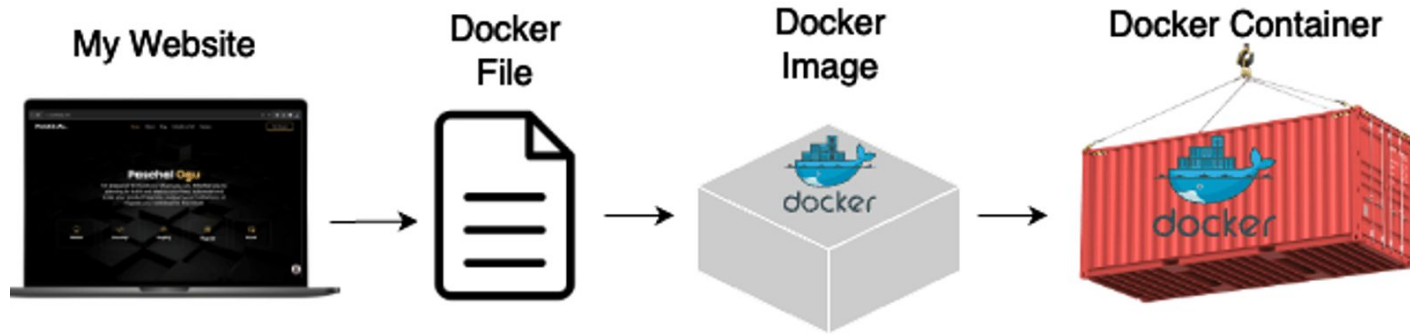


## Why Use Docker? : Avoid the “it works on my machine” problem



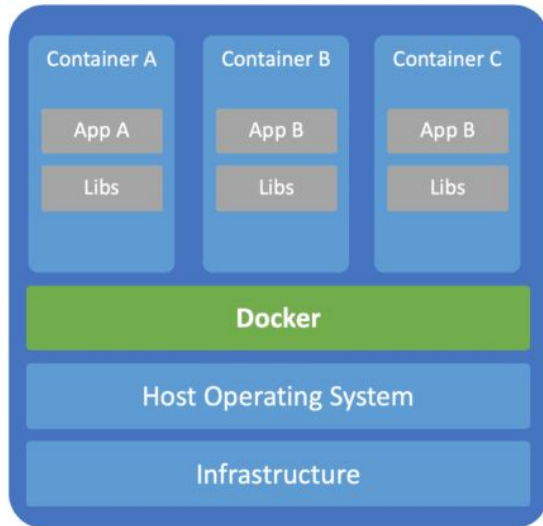
Daniel Stori {turnoff.us}

**Docker image** : Are **read-only** templates that contain instructions for creating a container

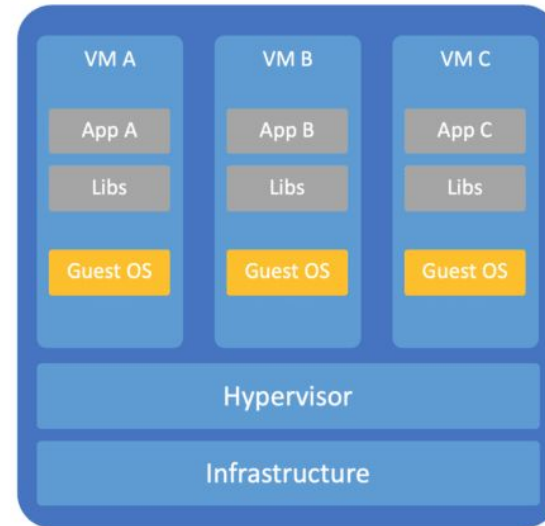


**Docker container image** : is a lightweight, standalone, executable package of software that includes everything needed to run an application : code, runtime, system tools, system libraries and settings

### Container



### Virtual Machines

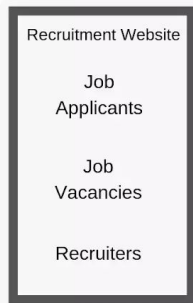


## What is Kubernetes ?

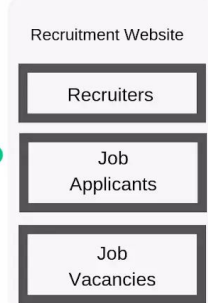
- A tool to manage many containers efficiently
- **Example** : Start, stop, or scale containers automatically when needed



### Monolithic Application



### Transition to Microservices



### Docker

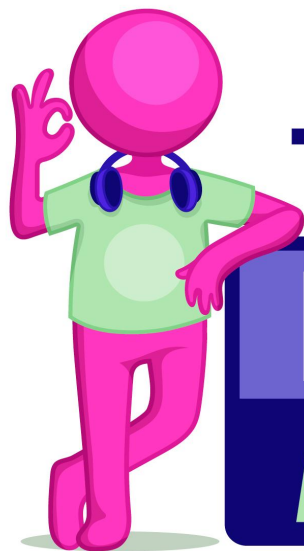
Create containers for your application



### Kubernetes

Launch your containerised application in K8s





**THANK YOU**  
**FOR YOUR**  
**ATTENTION**

